Submitted by:

Name: Kriti Pal
Registration No: 25BAI10713
Course: B.Tech AIML
Institute: VIT Bhopal University

📑 Project Report — Smart Revision Tool

1. Introduction

Revision is one of the most critical aspects of student learning, yet it is often inefficient. Students typically prepare long notes that are difficult to recall quickly before exams. This leads to stress, wasted time, and poor retention of key concepts.

To address this challenge, I developed the **Smart Revision Tool**, a Python-based program that automatically converts dense academic notes into concise flashcards using **Natural Language Processing (NLP)**. This project demonstrates how AIML concepts can be applied to solve real academic problems, making revision faster, structured, and more effective.

2. Problem Statement

- **Problem**: Students struggle to revise lengthy notes effectively.

- **Impact**: Inefficient revision reduces confidence and exam performance.
- **Objective**: Build a tool that automatically generates flashcards and mnemonics from notes, making revision structured and efficient.

This problem is highly relevant to my environment as a first-year B.Tech AIML student, where quick recall of technical and scientific concepts is essential.

3. Project Scope

The project covers the development of a text-to-flashcard pipeline using Python and NLTK. It is designed for English academic notes and focuses on sentence tokenization and keyword extraction.

Target Users:

- University students following flipped classroom models.
- Educators looking to generate quick quizzes from lecture materials.

4. Functional Requirements

- **Data Validation Module**: Checks input text for minimum length and content to ensure the system does not process empty or invalid data
- **NLP Processing Module**: Segments text into sentences and applies Part-of-Speech (POS) tagging to identify key nouns for question generation.
- **Flashcard Generation Module**: Uses POS tagging to identify key nouns and create fill-in-the-blank questions.
- **Export Module**: Automatically saves the generated Q&A pairs into a structured `.txt` file for offline study.

5. Non-functional Requirements

The project adheres to at least four non-functional requirements:

- **Maintainability:** The code is modularized into classes and functions to allow for future updates to the NLP logic.
- **Usability:** A simple command-line interface ensures that students can use the tool without advanced technical knowledge.
- **Reliability:** The tool includes an error-handling strategy to manage insufficient or poorly formatted input.
- **Resource Efficiency:** The program is lightweight, utilizing NLTK for fast processing on standard hardware or Google Colab

6. System Architecture

The system follows a pipeline architecture designed for NLP tasks:

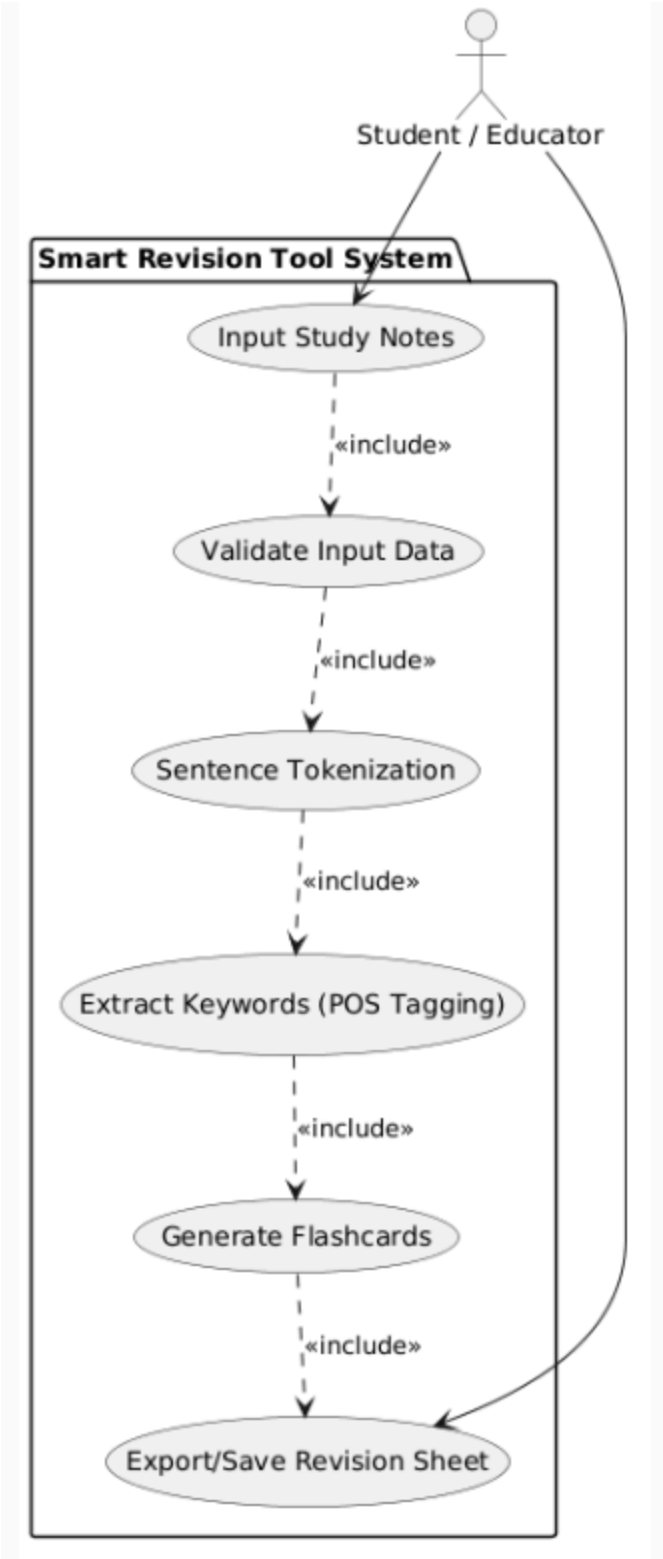
User Notes → Validation → Preprocessing → Flashcard Generation → Export/Output

7. Design Diagram

Start → Input Notes → Validate → Tokenize Sentences → Extract Keywords → Generate Q/A → Save to File → End

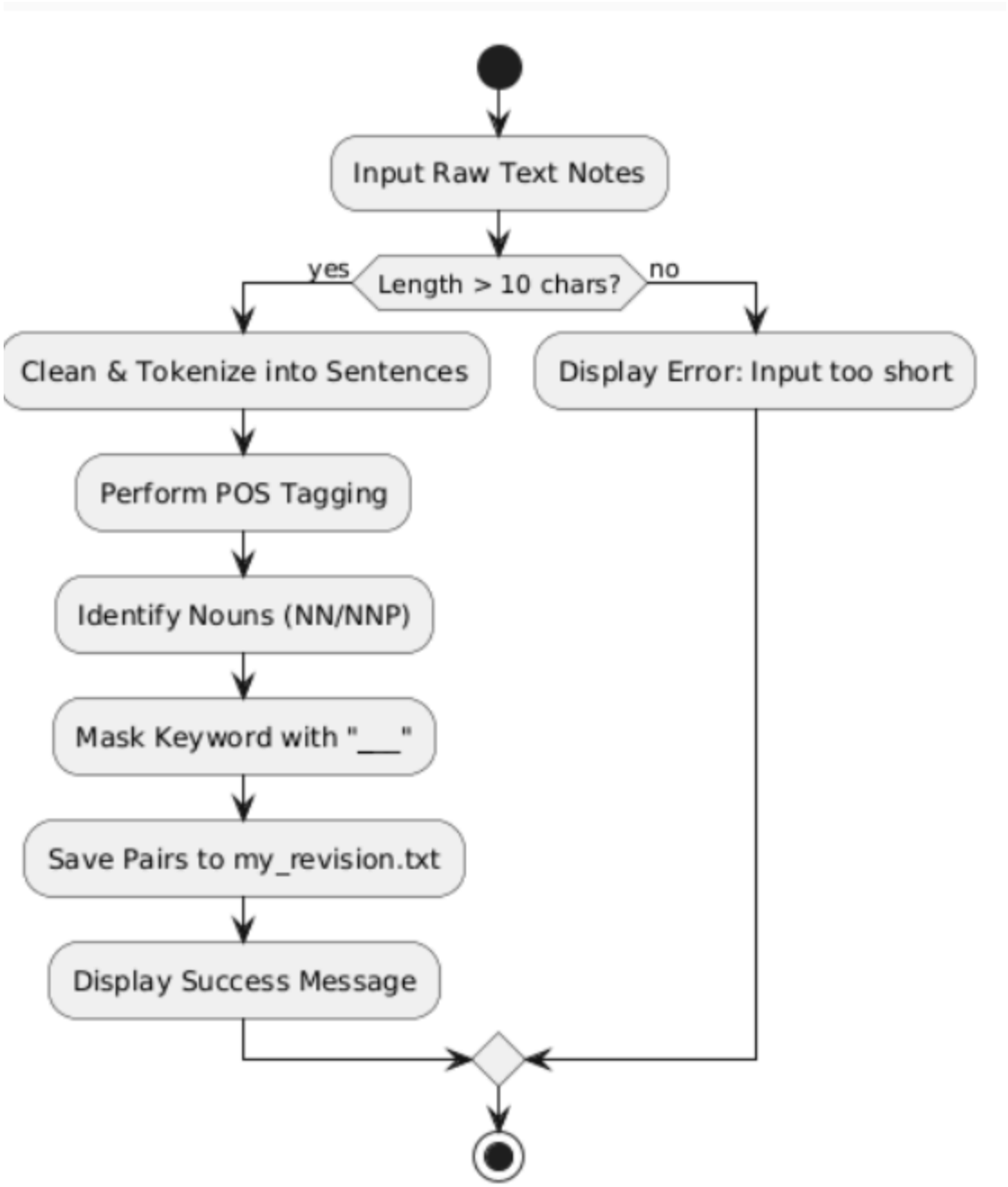
7.1 Use Case Diagram

- **Actor:** Student
- **Use Cases:**
  - Input Study Notes
  - Process Flashcards (System)
  - Download Revision Sheet



7.2 Workflow Diagram

1. **Start:** User provides text notes.
2. **Validation:** Is text > 10 characters? If no, show error.
3. **Tokenization:** Split text into sentences using sent_tokenize.
4. **Extraction:** Identify Nouns (NN/NNP) using pos_tag.
5. **Generation:** Mask the key noun with "_____" to create a question.
6. **Export:** Save results to my_revision.txt and End.



8. Design Decisions & Rationale

- **Choice of Python & NLTK:** Selected for simplicity and a strong ecosystem for text processing.
- **POS Tagging over Slicing:** Instead of taking the first five words, the tool uses Part-of-Speech tagging to identify actual subjects (nouns), making questions more educationally relevant.
- **File-Based Output:** A `.txt` file export was chosen to ensure the tool is compliant with executable requirements and provides a tangible study asset.

9. Implementation Details

- **Environment**: Google Colab.
- **Key Functions**:
- `sent_tokenize`: Sentence segmentation.
- `pos_tag`: Keyword identification based on linguistic structure.
- `Counter`: Statistical selection of the most relevant keyword per sentence.
- **Modular Design**: Minimum 5 meaningful functions/methods are used to handle setup, validation, processing, and file I/O.

10. Testing Approach

- **Functional Test**: Successfully generated flashcards from sample science notes.

- **Validation Test**: Verified that providing an empty string triggers the "Input too short" error message.
- **Output Test**: Confirmed that the `my_revision.txt` file is correctly formatted and saved.

11. Challenges Faced

- **Noise in Data**: Common words (stop words) were initially selected as keywords; resolved by integrating an NLTK stop-word filter.
- **Complex Sentences**: Sentences with multiple subjects required a frequency-based counter to pick the best "answer" noun.

12. Future Enhancements

- **Mnemonic Generator**: Using AI to create acronyms from extracted keywords.
- **OCR Support**: Integration of image-to-text to process photos of handwritten notes.
- **Advanced Models**: Moving from rule-based NLTK to Large Language Models (LLMs) for complex question phrasing.

13. References

- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- NLTK Project Documentation. (2023). https://www.nltk.org/
- VITyarthi BYOP Guidelines. (2026).